

Adopt SD-6 feature macros into the C++20 working draft

Abstract

This paper proposes to adopt the SD-6 feature macros into the C++20 working draft. Merging the feature macros removes vendors' obstacles for implementing them, makes them more portable and causes no delays in vendors tracking the macros in their implementations. This proposal is very much separate from what we should do with SD-6 as far as C++17 goes. We should publish a revision of SD-6 covering C++17 features as we have published SD-6 thus far. This proposal should not affect that publication in any way.

Goal

Feature test macros are so we can adopt the latest C++ features *faster* and always track the *latest* standard even before all our compilers support it -- so they are a force for *unifying* (tracking the very latest C++) and *avoiding* dialects that exist today (where code gets stuck on older C++ versions as dialects because it must target the lowest common denominator of supported compilers).

We have plenty of users who already do feature detection and conditional compilation. Feature macros allow doing that portably and more simply than with the alternative techniques available today.

Rumination

The general problem we are trying to solve here is feature detection, and based on detection results, conditional compilation of source code. There have been suggestions that this should, instead of macros, be done with a new language facility. The problem there is that

- feature-testing macros work today, yesterday and tomorrow. I can write my feature-macro protected code that uses C++20 facilities, and even a C++03 compiler will happily accept that code without any problems.
- If I want to use such a hypothetical future facility, I need to detect whether it's available. I either run into a chicken-egg problem or I need to use feature macros. Sure, perhaps there would be fewer feature macros, but there still would be some.
- I have no reason to believe adding such language facilities is likely to succeed, or that such facilities are easily adopted by various tools that read C++ code. The preprocessor is already there, and can provide feature-testing for every user right now, without having to wait multiple years.
- Standardizing feature-testing macros does not prevent us from adding a non-macro facility for feature testing and conditional compilation. Feature-testing macros and a potential non-macro facility serve slightly different purposes with different trade-offs.

See the very end of this document for some strawman ideas on such a language facility.

Current implementation status

Looking at some implementations, we notice the following:

- gcc and libstdc++ implement both the language feature macros and the library feature macros fairly completely.
- clang implements the language feature macros, libc++ implements fairly few of the library feature macros.
- msvc and its library don't implement feature macros.

There are some indications that there would be a non-zero chance that it would be more palatable for Microsoft to implement the feature macros if they are provided in an official WG21-sanctioned document. Some of these indications are mentioned in [the Toronto discussion notes on P0697R0](#).

Why put it into the IS? Would not a TS be a better choice?

A while ago, I myself thought that putting the macros into an IS makes no sense, because we publish an IS every 3 years, and can put out TSes much faster. However, I realized that there's no particular difference; implementations can track IS working drafts, which we publish after every meeting, so there's not necessary any delay for implementations of the macros, and for users being able to use the macros when adopting new features aggressively.

Putting the macros into a 'real' standard document makes them more portable, and putting them into the IS makes them the most portable, assuming that implementations continue to strive for conformance.

Why not just continue to publish a separate document and hope for implementations to implement them?

As mentioned before, some vendors can justify implementing the macros better if they are specified in an actual standard document.

I know of some users (the Qt Base module of Qt in particular) that use the feature macros correctly (without forking the language or otherwise forming an actual dialect, and without duplicating implementations), and seem to prefer a feature macro approach to testing `__cplusplus` (which implementations are unlikely to bump before they are in complete conformance) or to testing features in an implementation-specific manner. Having the macros be actually required, such testing remains simple and becomes (much more) portable, compared to having to check what implementation is used and which version of it is used.

I have fairly little to add why we should have feature macros to begin with. When used correctly, they allow library writers and library users to aggressively opt in to C++ features before an implementation is fully conforming to a new standard version, and to do so in a much simpler manner than detecting features in implementation-specific ways. Implementations that don't want to help early adopters can provide all feature test macros at once, at the same time they bump `__cplusplus`, and the presence of those macros will still help developers who've written code for implementations that do provide the feature test macros early. The feature macros can most likely be abused in various ways, but the proper ways of using them are a clear improvement to me. The next section attempts to provide a mini-FAQ of the reasons why we should have feature macros in the IS.

Mini-FAQ

Q: Isn't the frequency of IS publications too low for feature macros in the IS to be useful?

A: No, because they can be updated in every working draft.

Q: But it doesn't make sense to put the macros into an IS. Conforming implementations will have the same values for the macros and they will implement all features, so just checking `__cplusplus` is enough.

A: In isolation, that's all true. But before `__cplusplus` is bumped, users can write code that detects and uses some features, and that code will continue to work even after `__cplusplus` is bumped with no source code changes.

Q: Aren't we encouraging subsetting and dialects?

A: Not really. What we are encouraging is a way for implementations to give users a way to adopt features early in a portable way, in a fashion that continues to work even after `__cplusplus` is bumped, and of course works before it's bumped.

Q: But don't feature macros lead into language splintering and a proliferation of configurations?

A: Based on deployment experience, no. The amount of configurations is the same. The amount of macro permutations is the same, or smaller. Yes, that can mean that some builds have a lesser feature set, but that's a trade-off the code author made when choosing to use feature macros rather than a vendor-specific mechanism. The point is that feature macros don't seem to increase the amount of configurations and permutations.

Q: But none of this really works unless implementations provide feature macros early, tracking the working drafts. The only thing we guarantee is that a conforming implementation provides the IS versions of the macros.

A: Correct, but as described in the previous answers, some implementations will track the draft macros, and (correctly-written) code written to use those will automatically work with implementations that didn't provide the draft macros but will provide the IS macros. And, again, that will happen without code changes on the user side. We don't guarantee that every implementation provides every step of a migration path, but we do guarantee that all of them provide the last step, and we know that some will provide the earlier steps, so users can write code that benefits automatically from the point when the final macros are enabled, and already reaped the benefits when more aggressive implementations enabled them prior to the IS.

Q: But isn't it possible that feature macros are not enough? An implementation can define a feature macro but have a buggy implementation of the feature, so implementation-specific work-arounds may still be necessary.

A: No, that's not what feature macros are for. They are to test whether the feature exists at all, not to detect bugs, which needs to be done much less frequently, and will still require compiler version tests. Let's illustrate what feature-testing macros are for and what they are not for:

	Feature presence test	Compiler bug detection	Compile once	Build variations
Feature-testing macros	Y	N	N	Y
Compiler-version macros (<code>__GNUC__</code> , <code>_MSC_VER</code>)	N	Y	N	Y
Platform/target macros (bit width, endianness, architecture)	N	N	N	Y
Header guards	N	N	Y	N
Modules	N	N	Y	Y

A new language facility

I has been suggested to me that one way to do feature detection would be to annotate declarations somehow when they are exported from a module, and to import only those declarations for which the condition in the annotation evaluates to true.

Separately from that, I can envision doing conditional compilation with a new kind of language block:

```
std::compile_if<constant expression goes here> {  
    // here goes code that uses the language facility  
    // that the constant expression detected  
} else { // the else block is completely optional  
    // here goes code that doesn't use the aforementioned  
    // language facility, because it's not available  
}
```

We might also get the desired effect with injection:

```
constexpr {  
    if (constant expression goes here) -> {  
        // here goes code that uses the language facility  
        // that the constant expression detected  
    } else -> { // the else block is completely optional  
        // here goes code that doesn't use the aforementioned  
        // language facility, because it's not available  
    }  
}
```

Whether any experimentation with a module facility has been done, I don't know. These ideas are not, to my knowledge, very baked. Having such facilities would certainly be great, but we can't know when they would appear, and they don't work with a C++03 compiler, or with a C++2a compiler that hasn't yet implemented all of C++20. Feature macros do.